AI for Lexical & Syntax Assistance

Student Name: Neelam Jashwanth
Roll Number: 24CSB0A49

PHASE 1: Exploration & Analysis (Weeks 1-2)

- **Week 1: Problem Definition & Scope**
  - **Activity:** Define the problem of cryptic syntax errors in standard compilers. Establish objectives to build an AI-based error corrector.
  - **Deliverable:** Problem Statement, Objectives, and Scope Document.
- **Week 2: Literature Survey**
  - **Activity**: Research existing tools (like Grammarly for code, Deep Fix) and ML models (RNN, LSTM, Transformers) used for code correction.
  - **Deliverable**: Literature Review Report & Technology Selection.

Phase 2: Design & Data Preparation (Weeks 3-5)

- **Week 3: System Requirement Analysis**
  - **Activity***:* Define software/hardware requirements (Python, TensorFlow/PyTorch). Identify input formats (C/Python code snippets).
  - **Deliverable**: SRS (Software Requirement Specification) Document.
- **Week 4: System Architecture Design**
  - **Activity:** Design the architecture diagram showing the flow: Input Code -> Tokenizer -> AI Model -> Error Detection -> Suggestion -> Output.
  - **Deliverable**: System Architecture Diagram & Data Flow Diagrams (DFD).

- **Week 5: Dataset Collection & Preprocessing**
    - **Activity:** Collect datasets of C/Python code. Create a "Buggy Code vs. Clean Code" dataset for training the AI.
    - **Deliverable:** Preprocessed Dataset (CSV/JSON format).

Phase 3: Core Implementation (Weeks 6-10)

- **Week 6: Tokenizer Implementation**
    - **Activity***:* Implement a custom tokenizer or use an existing one (like Byte-Pair Encoding) to break code into tokens for the AI model.
    - **Deliverable:** Tokenizer Python Script.
- **Week 7: AI Model Development (Initial)**
    - **Activity:** Build the initial Neural Network model (Seq2Seq or LSTM) to understand code structure.
    - **Deliverable:** Initial Model Code (Untrained).
- **Week 8: Model Training**
    - **Activity:** Train the model using the dataset created in Week 5. Monitor accuracy and loss.
    - **Deliverable:** Trained Model Weights & Training Logs.
- **Week 9: Error Detection Module**
    - **Activity:** Develop the logic to identify *where* the error is located in the code (Line number & Token).
    - **Deliverable***:* Error Detection Script.

- **Week 10: Correction Logic Implementation**
  - **Activity:** Implement the logic to suggest fixes (e.g., inserting ';', fixing 'whiles' to 'while').
  - **Deliverable:** Auto-Correction Module.

Phase 4: Integration & Testing (Weeks 11-13)

- **Week 11: Integration with Compiler Frontend**
  - **Activity:** Connect the AI model with a basic Python script that acts as a compiler interface.
  - **Deliverable:** Integrated Tool (Alpha Version).
- **Week 12: Testing & Validation**
  - **Activity:** Test with various tricky code snippets. Calculate accuracy (How many errors fixed correctly?).
  - **Deliverable:** Test Case Report & Accuracy Metrics.
- **Week 13: Optimization & Security Analysis**
  - **Activity***:* Optimize for speed. Ensure the tool doesn't suggest dangerous code.
  - **Deliverable**: Performance Analysis Report.

Phase 5: Conclusion (Week 14)

- **Week 14: Final Documentation & Demo**
  - **Activity:** Finalize the project report, record a demo video, and prepare for the viva.
  - **Deliverable:** Final Project Report, Source Code, and Demo Video.